# Python Test: Thought Process

Cody A. Reed
DDOT CS-12 Data Scientist Assessment

October 16, 2026

I started with the Python test because I thought it was the best starting point for becoming familiar with the permit data, and I do almost all of my work, including GIS and data visualization, in scripting languages. There is no data dictionary, so before downloading anything I read the occupancy permit layer's page on DDOT's REST service. It lists every field and its type, and it shows that each record is a point in Web Mercator coordinates, that dates are stored in Eastern time, and that the server returns at most 2,000 records per request. It also shows that the layer has no functional class, which is why the permits need to be enriched.

The layer supports SQL in its queries. I am comfortable with SQL, so I did as much of the filtering as I could on DDOT's server. I counted a record as 2026 if it was applied for or issued in 2026, and kept only Construction Staging Area and Mobile Crane Work Zone permits. To find the three utilities, I grouped the 2026 records by each company field. `ApplicantCompany` is never blank, names the three utilities with only six spellings, and always agrees with the owner and permittee fields, so I identified each utility by its list of applicant names. I kept only DC Water, PEPCO, and Washington Gas, the three utilities named in the request, and left out the telecom and cable companies that file the same permits. The result was 5,038 permit locations in 3,149 permits. All of them are Construction Staging Areas, because the layer has no Mobile Crane Work Zone permits from these utilities. About half of the locations have no permit number or issue date. These are applications that have not been issued, and I kept them because the request includes applications. Claude Opus 5.5 helped me set up the REST queries and the SQL summaries.

For the enrichment, I compared the two approaches in the instructions. The Master Address Repository (MAR) seemed more precise and possibly faster, but its API requires a key, and the linear referencing system (LRS) felt more intuitive to me: one point in, the nearest road out. On a small test sample, every location returned the street named in its address, so I chose the LRS and planned to deal with alleys afterward. Claude Opus 5.5 helped me test both services. I split the work into three numbered scripts that each save their result for the next one, and I wrote a small `here.py` helper, like R's `here` package, so the file paths work on Windows. Looking up the 3,364 unique coordinates took about eight minutes with eight requests at a time. Two lookups failed during a brief network error, so I added a retry step.

Before joining the FHWA functional class, I counted the roads by type and by functional class to understand how to proceed. Every alley had no functional class, and the few other roads without one (mostly driveways) seemed insignificant, so I decided that every location in the output must have a functional class. In a work setting I would ask DDOT how to treat these other road types. The LRS reports DC's own class codes. Converting them to FHWA codes would have worked for almost all of my locations, but not for every road in the city, because some ramps are coded differently. I

want code that does things right and is robust for future queries, so I joined the official FHWA class from DDOT's Roadway SubBlock layer with a SQL query on each road's `SubBlockKey`, then kept the roads that are not alleys and have a class. That left 4,890 locations in 3,033 permits.

To finish, I joined the roads back to the permit locations and found each permit's highest functional class, which is the lowest FHWA code among its locations (1 is Interstate and 7 is Local). The data stay at one row per location, as the request asks, and I converted the dates from milliseconds since 1970 to Eastern calendar dates so they are easier for the Associate Director to read. As a check, the matched road appears in the permit's own address for 98.5% of locations; most of the rest are at street corners, where the LRS returned the cross street. With more time, I would look up the nearest street for alley locations instead of leaving them out, and I would compare the LRS results with MAR to learn which approach is more precise, then turn that approach into a simple reusable function. Claude Opus 5.5 helped me write and check the scripts.